Name: NGO TUNG CHUONG

ID: s225586848

- **Part A**

The Stack data structure provides **Push** and **Pop**, the two operations to write and read data, respectively. Your task is to extend this data structure so that, in addition to these two operations, it also provides the **Min** operation to return the smallest element in the stack. Remember that the new data structure must operate in a constant $O(1)$ time for all three aforementioned operations.

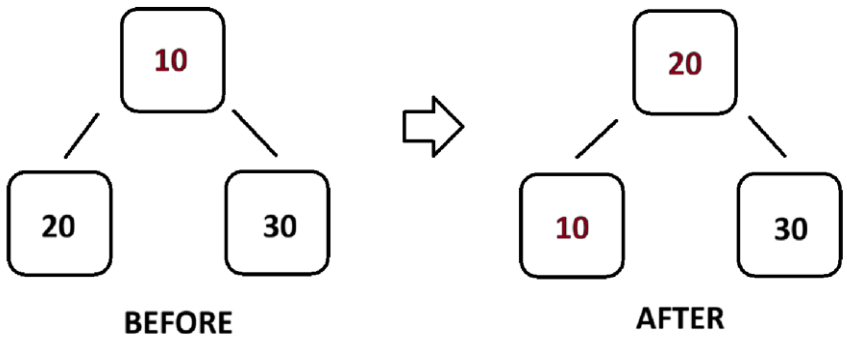
```
Function Push(x) {
    MainStack.Push(x);

    If (MinStack.IsEmpty() OR x <= MinStack.Peek())
        MinStack.Push x;
    else {
        MinStack.Push(MinStack.Peek());
    }
}
Pop() – remain the same
Min() {
    Peek();
}

Function Pop() {
    MinStack.Pop();    // help two stack sync
    Return MainStack.Pop();
}

Function Min() {
    Return MinStack.Peek();
}
```

- **Part B**

After reading about the Binary Max-Heap data structure, Elon Musk decided to implement it as part of his supper-duper algorithm. However, as he was not paying enough attention in the class, his implementation of the **DownHeap** procedure (also known as the **Max-Heapify**) is not correct. He gave us the following pseudocode of his implementation.

```
Function DownHeap( index ):
    If ( H[Left( index )].key > H[index].key )
    {
        Swap elements H[index] and H[Left( index )]
        DownHeap ( Left( index ) )
    }
    Else If ( H[Right( index )].key > H[index].key )
    {
        Swap elements H[index] and H[Right( index )]
        DownHeap ( Right( index ) )
    }
```

Your task is to give a numeric example of a heap H, for which the call of the above **DownHeap** procedure leads to a wrong structure of H as a max-heap. Specifically, you need to propose a heap where the root $H[1]$ is the only node breaking the essential **Max-Heap Property** so that running the above **DownHeap(1)** (i.e., index = 1) does NOT result in a valid max-heap. You must draw the heap H before and after running **DownHeap(1)**. Note that your instance can be small.



- **Part C**

Assume now that Elon Musk has eventually fixed the problem above and developed a correct implementation of the Max-Heap that provides all the standard operations such as:

- **DeleteMax**, **Insert(key)**, **IncreaseKey(index, key)** in a logarithmic $O(\log n)$ time, and
- **Max** in a constant $O(1)$ time,

where n is the total number of stored elements. Elon Musk wants to add a new capability to his max-heap. Specifically, he would like to implement a **DeleteElement(index)** operation that removes element H[index] from the max-heap H consisting of n elements in a logarithmic $O(\log n)$ time. Note that the H[index] does

NOT need to be the maximum-key element in H. Also, naturally, H must maintain the Max-Heap Property after deletion of H[index].

Your task is to help Elon Musk by providing a pseudocode of such a **DeleteElement(index)** operation accompanied with a brief analysis of its running time as a function of n elements.

Pseudocode:

```
Function DeleteElement(index) {
    if (index <  1 OR index > H.size) return ERROR;  // base case
    IncreaseKey(index, int.MaxValue); // we make the element at index
                                      // become largest number in Heap Tree
    DeleteMax();
}
```

- **Part D**

Elon Musk is cooking cupcakes. Help him to design an algorithm with complexity better than $O(n \log n)$ that finds the k-th heaviest cupcake.

I will apply the "Partition" logic of Selection Algorithm of QuickSort. Instead of sorting both sides of a pivot, it only recurses into the partition that contains the kth largest element. With the average time complexity of O(n). Which better than O(nlogn).

Where I take the inspiration from: Selection Algorithms - GeeksforGeeks